R3.05 · TD-TP 01

PROGRAMMATION SYSTÈME

Le langage C — partie 1

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Bienvenue au LAMAE, recrue. Avant de toucher au noyau, il faut obtenir ton badge d'accès : six épreuves, six drapeaux à capturer. Ton seul outil : un terminal, gcc et ta logique.

OBJECTIFS DE LA SÉANCE

- ☐ Compiler et exécuter un programme C sous WSL avec les bonnes options de `gcc`.
- ☐ Comprendre les 4 étapes de la chaîne de compilation (préprocesseur, compilation, assemblage, édition de liens).
- ☐ Manipuler les types simples (`char`, `int`, `long`, `double`) et la table ASCII.
- ☐ Afficher avec `printf` et saisir avec `scanf` **sans** déborder un tableau.
- ☐ Écrire des branchements (`if`, `switch`) et des boucles (`while`, `for`).

FIL ROUGE — Opération « Badge LAMAE »

Chaque mission te fait gagner un **drapeau** de la forme `FLAG{...}`. Note-les au fur et à mesure dans le fichier `badge.txt` : l'enseignant·e les vérifiera en fin de séance. Les exercices sont cotés de ★ (incontournable) à ★★★ (pour aller loin). Les encadrés rouges **Sécurité** ne sont pas optionnels : un·e agent·e du LAMAE ne laisse jamais traîner un débordement de tampon.

Mission	Contenu	Durée
0 · Mise en route	WSL, VS Code, premier programme, options de `gcc`	15 min
1 · La forge	Les 4 étapes de la compilation	15 min
2 · Le code secret	`char`, ASCII, `printf` et ses formats	25 min
3 · Le guichet	`scanf`, tableaux de caractères, débordements	25 min
4 · Le thermomètre	Boucles et division entière	15 min
5 · Le coffre	Jeu « devine le nombre », `switch`	25 min
Bonus	Pour les rapides : dépassement d'entier	—

RÈGLE D'OR DE LA SÉANCE

Compile **toujours** avec les avertissements activés. Un avertissement de `gcc` est un bug qui n'a pas encore explosé.

```
• • • compilation recommandée en R3.05
$ gcc -std=c17 -Wall -Wextra -pedantic -g prog.c -o prog
$ ./prog
```

Mission 0 — Mise en route

Comme en première année, vous travaillez dans le shell Linux de **WSL** (Windows Subsystem for Linux), depuis **Visual Studio Code**.

1. Créez un dossier de travail `r305/tp01` (évitez les espaces et les accents dans les noms de dossiers : ils compliquent la vie dans le shell).
2. Dans VS Code, ouvrez un terminal : menu **Terminal › Nouveau terminal** (raccourci **Ctrl+ù** sur clavier AZERTY), puis tapez `wsl` (ou `ubuntu`).
3. Placez-vous dans votre dossier avec `cd` et vérifiez avec `pwd` (chemin absolu du répertoire courant). Depuis WSL, le disque Windows `C:` est visible sous `/mnt/c/`.
4. Vérifiez que gcc est installé avec `gcc --version`. Si la commande est introuvable, installez-le comme indiqué ci-dessous.

```
• • • installer gcc (une seule fois)

$ sudo apt update                          # met à jour la liste des paquets
$ sudo apt install gcc build-essential     # gcc, make et les bibliothèques C
$ sudo apt install gdb valgrind            # débogueur et détecteur de fuites
$ gcc --version                            # vérification
gcc (Ubuntu 13.3.0) 13.3.0
```

`sudo` demande votre mot de passe Linux (celui choisi à l'installation de WSL) ; rien ne s'affiche pendant que vous le tapez, c'est normal.

EX 0 — Hello, LAMAE !

★ 5 min

Saisissez le programme suivant dans `hello.c`, compilez-le puis exécutez-le.

```
• • • hello.c

#include <stdio.h>   // bibliothèque standard d'entrées-sorties (printf)

int main(void)       // main renvoie un int au système
{
    printf("Hello, LAMAE !\n");
    return 0;        // 0 = succès, lu par le shell avec echo $?
}
```

```
• • • TERMINAL

$ gcc -std=c17 -Wall -Wextra -pedantic -g hello.c -o hello
$ ls
hello  hello.c
$ ./hello
Hello, LAMAE !
$ echo $?
0
```

- Sans l'option `-o hello`, l'exécutable s'appelle `a.out`.
- Pourquoi faut-il taper `./hello` et pas `hello` ? Affichez la variable `PATH` (`echo $PATH`) : le shell ne cherche les programmes que dans ces répertoires, et le répertoire courant `.` n'en fait pas partie (c'est volontaire : sinon un programme piégé nommé `ls` déposé dans un dossier serait exécuté à la place du vrai).
- Modifiez `return 0;` en `return 42;`, recompilez et affichez `echo $?`. **Drapeau :** `FLAG{` suivi de la valeur affichée, suivi de `}`.

> **PIÈGE CLASSIQUE** **Le tiret qui n'en est pas un**
> Si vous copiez une commande depuis un PDF ou un traitement de texte, le tiret de `-o` peut devenir un tiret long (`–o`) : gcc le prend alors pour un nom de fichier. Tapez vos commandes au clavier.

Option de gcc	Rôle
`-std=c17`	Compiler selon la norme C17 (ISO/IEC 9899:2018), sans extension implicite.
`-Wall -Wextra`	Activer la quasi-totalité des avertissements utiles.
`-pedantic`	Signaler tout ce qui n'est pas du C standard.
`-g`	Ajouter les informations de débogage (pour `gdb` et `valgrind`).
`-o nom`	Nommer l'exécutable produit.
`-fsanitize=address,undefined`	Instrumenter le programme pour détecter à l'exécution débordements et comportements indéfinis (utile en mise au point).

Mission 1 — La forge : de la source à l'exécutable

Le C est un langage **compilé** : le fichier source (texte) doit être traduit en langage machine. `gcc` enchaîne en réalité quatre étapes :

Étape	Produit	Ce qui se passe
Préprocesseur	`.i`	Transformations purement textuelles : inclusion des `#include`, remplacement des `#define`, suppression des commentaires.
Compilation	`.s`	Traduction en **assembleur** (mnémoniques lisibles des instructions du processeur).
Assemblage	`.o`	Traduction en binaire : le **fichier objet**.
Édition de liens	exécutable	Réunion des fichiers objets et des bibliothèques (`libc` : `.so` partagées, `.a` statiques).

EX 1 Ouvrir la forge, étape par étape ★ 15 min

Sur `hello.c`, arrêtez gcc après chaque étape et observez :

```
• • •  TERMINAL
$ gcc -E hello.c -o hello.i      # préprocesseur seulement
$ wc -l hello.c hello.i
$ gcc -S hello.c -o hello.s      # jusqu'à l'assembleur
$ gcc -c hello.c -o hello.o      # jusqu'au fichier objet
$ file hello.o hello
$ nm hello.o                     # symboles : qui est défini, qui manque ?
$ gcc hello.o -o hello           # édition de liens
```

1. Combien de lignes contient `hello.i` ? D'où viennent-elles ?
2. Cherchez `printf` dans `hello.s`. Surprise : gcc a peut-être appelé `puts` à la place. Pourquoi ce remplacement est-il possible ici ? (Indice : que fait `printf` d'une chaîne sans `%` terminée par `\n` ?)
3. Dans la sortie de `nm hello.o`, que signifient les lettres `T` et `U` devant `main` et `puts` ? Qui fournit `puts` au moment de l'édition de liens ?
4. **Drapeau :** `FLAG{` + le nom de la fonction effectivement appelée dans `hello.s` + `}`.

> **CULTURE** **Un peu d'histoire**
>
> Le C a été conçu au début des années 1970 par **Dennis Ritchie** aux Bell Labs, pour réécrire le système **UNIX** (créé avec Ken Thompson) sur un DEC PDP-11. En 1978, Kernighan et Ritchie publient *The C Programming Language* (le « K&R »). Le langage est normalisé par l'ANSI en 1989 (C89), repris par l'ISO en 1990 (C90), puis révisé : **C99**, **C11** (qui supprime `gets`), **C17** (norme utilisée en R3.05) et **C23**.

Conventions de codage du module

Ces conventions facilitent la lecture et la réutilisation de votre code par d'autres développeurs.

Élément	Règle	Exemples
Constantes	Majuscules, mots séparés par `_`	`MAX`, `NB_LIGNE`, `NB_COLONNE`
Variables et fonctions	Mots accolés, majuscule à chaque mot sauf le premier ; un verbe pour une fonction	`enregistreListe()`, `prixUnitaire`
Types construits	Majuscule initiale	`struct Personne`, `typedef struct Personne Employe;`
Commentaires	Pour le lecteur : le *pourquoi*, pas le *comment*	`// ...` ou `/* ... */`
Indentation	Un niveau (4 espaces ou une tabulation) par bloc imbriqué	voir ci-dessous

```c
int tab[NB_LIGNE][NB_COLONNE];
for (int i = 0; i < NB_LIGNE; i++) {
    for (int j = 0; j < NB_COLONNE; j++) {
        if (i == j)
            tab[i][j] = 0;
        else
            tab[i][j] = i + j;
    }
}
```

Mission 2 — Le code secret : types, ASCII et printf

En C, un `char` est un **petit entier** (1 octet). Tout `char` peut être utilisé dans une expression entière : si `c` vaut `'A'`, l'expression `c + 1` désigne le caractère suivant dans la table **ASCII** (`man ascii` dans le terminal pour l'afficher). Le programme suivant affiche donc `B`.

```c
#include <stdio.h>

int main(void)
{
    char c = 'A';          // 'A' vaut 65 en ASCII
    printf("%c\n", c + 1); // affiche le caractère de code 66 : B
    return 0;
}
```

La fonction printf

`printf` est une fonction d'impression **formatée** : `printf("chaîne de contrôle", expr1, ..., exprN);`. La chaîne de contrôle contient le texte et une **spécification de format** (introduite par `%`) pour chaque expression.

Format	Type attendu	Écriture
`%d` / `%i`	`int`	décimale signée
`%ld`	`long`	décimale signée
`%u` / `%lu`	`unsigned int` / `unsigned long`	décimale non signée
`%o` / `%lo`	`unsigned int` / `unsigned long`	octale
`%x` / `%lx`	`unsigned int` / `unsigned long`	hexadécimale
`%zu`	`size_t` (résultat de `sizeof`)	décimale non signée
`%f`	`double` (un `float` est promu en `double`)	virgule fixe
`%e`	`double`	notation exponentielle
`%g`	`double`	la plus courte de `%f` et `%e`
`%Lf` `%Le` `%Lg`	`long double`	idem, en `long double`
`%c`	`int` (converti en caractère)	caractère
`%s`	`char *` (chaîne terminée par `'\0'`)	chaîne de caractères
`%p`	`void *`	adresse (pointeur)
`%%`	—	le caractère `%` lui-même

EX 2 Décrypter le message

★ 10 min

Le LAMAE a intercepté une suite de codes. Écrivez un programme qui affiche, avec `%c`, les caractères de codes décimaux **70 76 65 71 123 65 83 67 73 73 125**, puis la même chose décalée de +1 (chaque code + 1). Lequel des deux est le drapeau ?

> **INDICE**
> Pas besoin de tableau : une série de `printf("%c", 70);` suffit. Pour les plus à l'aise, la boucle viendra en mission 4.

EX 3 Le grand test des formats

★★ 15 min

Récupérez `formats.c` (dossier `code/sujet01/`) et compilez-le **avec** `-Wall -Wextra`.

formats.c (extrait)

```c
int i = 23674;
int j = -23674;
long k = (1L << 32);
double x = 1e-8 + 1000;
char c = 'A';
char *chaine = "chaine de caracteres";

printf("%d \t %u \t %o \t %x\n", i, i, i, i);
printf("%d \t %u \t %o \t %x\n", j, (unsigned)j, (unsigned)j, (unsigned)j);
printf("%d \t %o \t %x\n", k, k, k);          // volontairement faux !
printf("%ld \t %lu \t %lo \t %lx\n", k, (unsigned long)k, (unsigned long)k, (unsigned long)k);
printf("%f \t %e \t %g\n", x, x, x);
printf("%.2f \t %.2e\n", x, x);
printf("%.20f \t %.20e\n", x, x);
printf("%c \t %d\n", c, c);
printf("%s \t %.10s\n", chaine, chaine);
```

1. Lisez les avertissements de gcc : quelle ligne est signalée et pourquoi ? Que s'affiche-t-il pour cette ligne ? Est-ce garanti ?
2. Pourquoi `-23674` affiché en `%u` donne-t-il un nombre aussi grand ? (Pensez au complément à deux sur 32 bits.)
3. `%.20f` affiche des chiffres « faux » au bout de `x`. Expliquez (un `double` a environ 15-16 chiffres significatifs).
4. Complétez le programme pour afficher, avec `%zu`, la taille en octets (`sizeof`) de `char`, `int`, `long`, `float`, `double` et `char *` sur votre machine. Remplissez le tableau :

`char`	`int`	`long`	`float`	`double`	`char *`

Drapeau : `FLAG{` + `sizeof(long)` + `}`.

> **PIÈGE CLASSIQUE** **Format et type doivent correspondre**
>
> Donner à `printf` un `long` pour un `%d` (ou un `int` pour un `%s` !) est un **comportement indéfini** : le programme peut afficher n'importe quoi, ou planter. Le compilateur ne peut vous prévenir que si vous activez `-Wall`, d'où la règle d'or.

Mission 3 — Le guichet : saisir sans déborder

La fonction `scanf` lit des données au clavier et les range **aux adresses** passées en argument : `scanf("chaîne de contrôle", &var1, ..., &varN);`. Ses formats ressemblent à ceux de `printf` (attention : pour un `double`, c'est `%lf` en saisie). Les données saisies sont séparées par des blancs ou des retours à la ligne. On peut limiter la largeur lue : `%9s` lit **au plus 9 caractères** (et en écrit 10 avec le `'\0'` final).

C

```c
#include <stdio.h>

int main(void)
{
    int i;
    printf("Entrez un entier en hexadécimal : i = ");
    if (scanf("%x", (unsigned *)&i) != 1) {   // scanf renvoie le nombre de conversions réussies
        fprintf(stderr, "Saisie invalide\n");
        return 1;
    }
    printf("i = %d\n", i);   // 1a  ->  i = 26
    return 0;
}
```

Le `&` signifie « adresse de » : `scanf` doit savoir **où** écrire. Nous y reviendrons longuement avec les pointeurs (sujet 03).

EX 4 Le guichet du magasin

★ 10 min

Écrivez un programme qui demande le **nom** d'un produit et son **prix**, puis les affiche : `Produit : Sabre laser — Prix : 129.90 €`. Pour le nom, utilisez un tableau de caractères :

```c
#define TAILLE_NOM 10
char nom[TAILLE_NOM];   // 10 octets : 9 caractères + le '\0' final
double prix;
```

Testez votre programme avec le nom `Faucon` puis avec le nom `FauconMillenium-Edition-Collector`.

> **SÉCURITÉ Le débordement de tampon (buffer overflow)**
>
> Avec `scanf("%s", nom)`, rien n'empêche l'utilisateur de taper 40 caractères dans un tableau de 10 : les octets en trop écrasent **les variables voisines en mémoire** (ici, peut-être `prix`), voire l'adresse de retour de la fonction. C'est la faille la plus célèbre de l'histoire de la sécurité (ver Morris en 1988, et des milliers d'autres depuis).
>
> 1. Recompilez avec `-fsanitize=address -g`, relancez avec le nom long : lisez le rapport `stack-buffer-overflow`.
> 2. Corrigez de deux façons : avec une largeur `scanf("%9s", nom)`, puis avec `fgets(nom, sizeof nom, stdin)` (qui lit aussi les espaces ; pensez à retirer le `'\n'` final).
> 3. Vérifiez la valeur de retour de `scanf` pour le prix : que se passe-t-il si l'on tape `abc` ?
>
> **Jamais** `gets` : cette fonction ne connaît pas la taille du tableau, elle a été **retirée du langage en C11**.

EX 5 Le piège du format

★★ 10 min

Un stagiaire a écrit `printf(nom);` pour afficher le nom saisi. Testez cette ligne en saisissant comme nom `%x.%x.%x`, puis `%s%s%s%s`. Qu'observez-vous ? Que lit `printf` quand il trouve un `%` sans argument correspondant ?

> **SÉCURITÉ Faille « format string »**
>
> Une donnée venant de l'utilisateur ne doit **jamais** servir de chaîne de format. Écrivez toujours `printf("%s", nom);` ou `fputs(nom, stdout);`. gcc vous prévient avec `-Wformat-security` (inclus dans `-Wall` sur la plupart des distributions). Le guide ANSSI en fait une règle explicite.

Drapeau : `FLAG{` + le nom de la fonction de saisie de ligne interdite depuis C11 + `}`.

Mission 4 — Le thermomètre : branchements et boucles

Branchements

```c
if (expression1)
    instruction1
else if (expression2)
    instruction2
/* ... autant de else if que nécessaire ... */
else
    instructionN          // le dernier else est facultatif

switch (expression) {     // expression entière
    case CONSTANTE_1:
        instructions;
        break;            // sans break, on « tombe » dans le case suivant
    case CONSTANTE_2:
        instructions;
        break;
    default:              // facultatif : aucune constante ne correspond
        instructions;
        break;
}
```

Boucles

```c
int i = 1;
while (i < 10) {          // tant que la condition est vraie (non nulle)
    printf("i = %d\n", i);
    i++;
}

for (int i = 0; i < 10; i++)   // initialisation ; condition ; incrément
    printf("i = %d\n", i);
```

> **PIÈGE CLASSIQUE** **= ou == ?**
>
> `if (x = 0)` **affecte** 0 à `x` et la condition est toujours fausse. gcc avec `-Wall` affiche « suggest parentheses around assignment used as truth value ». Encore une raison de lire les avertissements.

EX 6 — Table des températures

★ 15 min

Affichez la table des températures en degrés Fahrenheit et leur équivalent en degrés Celsius, de 0 °F à 300 °F par pas de 20 °F, **sans tableau**.

1. Calculez avec la formule `celsius = 5 * (fahr - 32) / 9;` (variables `int`). Comparez avec une calculatrice : pourquoi est-ce différent ?
2. Écrivez la formule `celsius = (5 / 9) * (fahr - 32);`. Que se passe-t-il ? Pourquoi ?
3. Écrivez `celsius = (5.0 / 9) * (fahr - 32);` avec `celsius` de type `double` et un format `%6.1f`. Alignez joliment les colonnes.
4. Demandez enfin une valeur en °F à l'utilisateur (en vérifiant le retour de `scanf`) et affichez sa conversion.

> **INDICE**
> `5 / 9` est une **division entière** entre deux `int` : son résultat est tronqué vers zéro. Il suffit qu'un des deux opérandes soit un `double` pour obtenir une division réelle.

Drapeau : `FLAG{` + la valeur de `(5 / 9) * 100` calculée en C + `}`.

Mission 5 — Le coffre : devine le nombre

Le coffre du LAMAE s'ouvre avec un nombre secret entre 0 et 99. Vous avez **10 essais**.

EX 7 Ouvrir le coffre

★ 15 min

Codez le jeu : le programme tire au hasard un entier entre 0 et 99, puis l'utilisateur propose des valeurs ; le programme répond « trop grand », « trop petit » ou « coffre ouvert ! », et s'arrête après 10 essais ratés.

```c
#include <stdlib.h>   // srand, rand
#include <time.h>     // time
/* ... */
srand((unsigned) time(NULL));        // graine différente à chaque exécution
int valeurATrouver = rand() % 100;   // entre 0 et 99
```

> **PIÈGE CLASSIQUE** **La saisie qui boucle à l'infini**
> Si l'utilisateur tape `abc`, `scanf("%d", &n)` renvoie 0 et **laisse** `abc` dans le tampon d'entrée : le `scanf` suivant échoue de nouveau, indéfiniment. Testez la valeur de retour et videz la ligne : `int c; while ((c = getchar()) != '\n' && c != EOF) { }`. Remarquez le `int c` : `getchar` renvoie un `int` pour pouvoir distinguer `EOF` de tous les caractères.

EX 8 Niveaux de difficulté

★★ 10 min

Ajoutez un menu (avec `switch`) qui propose : **1** — facile (0 à 99, 10 essais), **2** — difficile (0 à 999, 10 essais), **3** — expert (0 à 9999, 14 essais), **0** — quitter. Rejouez tant que l'utilisateur ne choisit pas 0.

1. Quelle stratégie garantit de trouver un nombre entre 0 et 999 en 10 essais au plus ? Combien d'essais faut-il au pire pour 0 à 9999 ?
2. Faites jouer l'ordinateur : écrivez une fonction qui trouve le nombre par **dichotomie** et affiche le nombre d'essais.

> **CULTURE** **rand() n'est pas un générateur sûr**
> `rand()` est prévisible : connaissant quelques sorties, on retrouve la graine (et `time(NULL)` se devine facilement !). Il convient pour un jeu, **jamais** pour un mot de passe, un jeton de session ou une clé. Sous Linux, on utilise `getrandom(2)` ou `/dev/urandom`. Par ailleurs `rand() % 100` n'est pas parfaitement uniforme (biais du modulo).

Drapeau : `FLAG{` + le nombre d'essais nécessaires au pire, par dichotomie, pour 0 à 999 + `}`.

POUR LES RAPIDES **Le compteur qui déborde ★★★**

1. Calculez et affichez `n!` pour n de 1 à 25 avec une variable `int`, puis `long`, puis `unsigned long long` (`%llu`). À partir de quel n le résultat devient-il faux ? Pourquoi peut-il devenir négatif ?
2. Recompilez avec `-fsanitize=undefined` : que signale l'exécution ? (Le dépassement d'un entier **signé** est un comportement indéfini en C ; celui d'un non signé « boucle » modulo 2^n.)
3. Avant chaque multiplication, testez si `resultat > LONG_MAX / n` (`<limits.h>`) pour arrêter proprement le calcul. C'est exactement ce que recommande le guide ANSSI pour les calculs de tailles.
4. Écrivez une fonction `afficherBinaire(unsigned int x)` qui affiche les 32 bits de `x` (opérateurs `>>` et `&`). Affichez `23674` et `(unsigned)-23674`.

Bilan de la séance

À RETENIR

- On compile **toujours** avec `-std=c17 -Wall -Wextra -pedantic -g`, et on lit les avertissements.
- `int main(void)` renvoie un `int` : `return 0;` en cas de succès.
- Un `char` est un entier ; `getchar` renvoie un `int`.
- Le format de `printf`/`scanf` doit correspondre exactement au type de l'argument.
- Toute saisie dans un tableau doit être **bornée** (`%9s`, `fgets`) et toute valeur de retour de `scanf` **testée**.
- Jamais de donnée utilisateur comme format : `printf("%s", s)`.

- ☐ J'ai mes 6 drapeaux dans `badge.txt` (missions 0, 1, 2, 3, 4, 5).
- ☐ Mes programmes compilent sans **aucun** avertissement.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 3 printf`, `man 3 scanf`	Liste complète des formats, modificateurs et valeurs de retour (section 3 = bibliothèque C).
`man ascii`, `man gcc`	La table ASCII ; toutes les options du compilateur.
cppreference — C	Référence du langage et de la bibliothèque standard, norme par norme (C89 à C23).
Kernighan & Ritchie, *Le langage C* (2e éd., Dunod)	Le livre fondateur, court et dense.
J.-M. Rifflet, *La programmation sous UNIX*	Référence francophone pour la suite du module (système).
Guide ANSSI, *Règles de programmation pour le développement sécurisé de logiciels en langage C* (dossier du CM)	Règles sur les options de compilation, les entiers, les chaînes de caractères.
Compiler Explorer	Voir instantanément l'assembleur produit par gcc pour un bout de C.
`gdb`	Exécuter pas à pas : `gdb ./prog`, puis `break main`, `run`, `next`, `print variable`.

Pour approfondir

CULTURE **Comportement indéfini (undefined behavior)**

La norme C laisse certaines situations « indéfinies » : débordement d'un `int`, accès hors d'un tableau, mauvais format `printf`, variable non initialisée... Le compilateur a alors **le droit de tout faire**, y compris supprimer un test qu'il juge inutile. Les options `-fsanitize=undefined` et `-fsanitize=address` détectent beaucoup de ces cas à l'exécution : prenez l'habitude de les utiliser pendant la mise au point.

CULTURE **Pourquoi `main` renvoie-t-il un int ?**

La valeur renvoyée par `main` (ou passée à `exit`) devient le **code de retour** du processus, que le shell lit avec `$?` et que les scripts testent (`./prog && echo ok`). Par convention, 0 signifie succès ; `EXIT_SUCCESS` et `EXIT_FAILURE` sont définis dans `<stdlib.h>`. Vous verrez au sujet 06 comment un processus père récupère ce code avec `wait`.